Advisory boards aren't only for executives. Join the LogRocket Content Advisory Board today →

Jul 29, 2025 · 9 min read

Build interactive React UIs for LLM outputs using llm-ui

Emmanuel John
I'm a full-stack software developer, mentor, and writer. I am an open source enthusiast. In my spare time, I enjoy watching sci-fi movies and cheering for Arsenal FC.

Table of contents

Why use llm-ui

Project setup

Install llm-ui

Core concepts

Setting up the UI

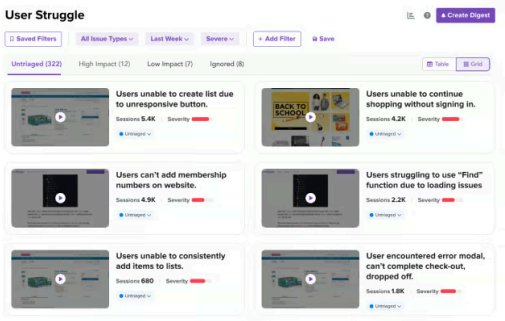
`Block` display component

Getting LLM stream

Integrating the user interface

Comparing llm-ui and its alternatives

Conclusion



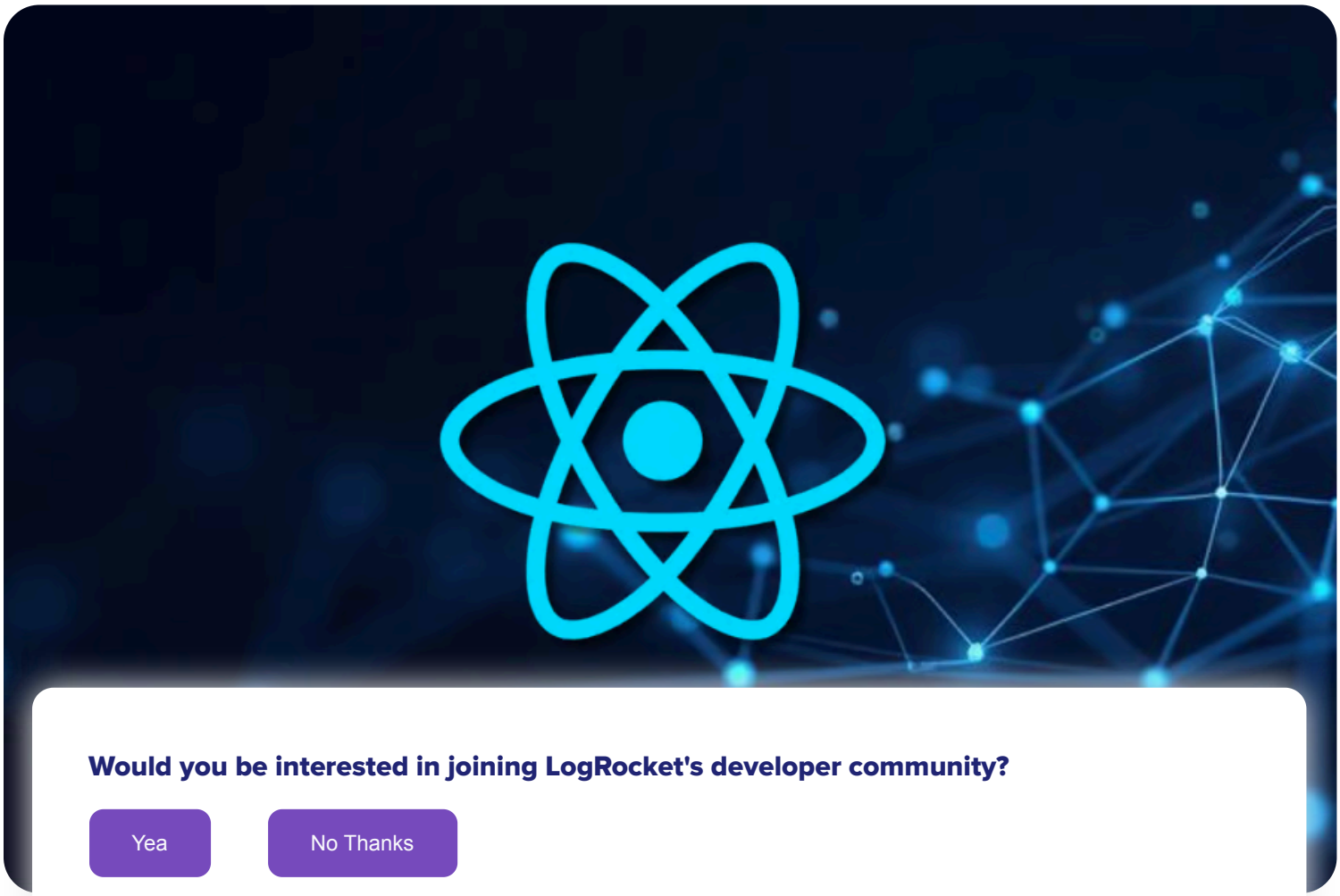
Introducing Galileo AI
LogRocket's Galileo AI watches every session, surfacing impactful user struggle and key behavior patterns.
LEARN MORE

See how LogRocket's Galileo AI surfaces the most severe issues for you

No signup required

Check it out

We've all seen the impressive text generated by large language models (LLMs), but simply displaying that raw output isn't always user-friendly. The real challenge is turning those responses into clean, structured, and interactive interfaces that enhance the user experience.



Would you be interested in joining LogRocket's developer community?

Yea

No Thanks

This tutorial introduces `llm-ui`, a flexible React library that helps you build rich UIs around your LLM's streaming output.

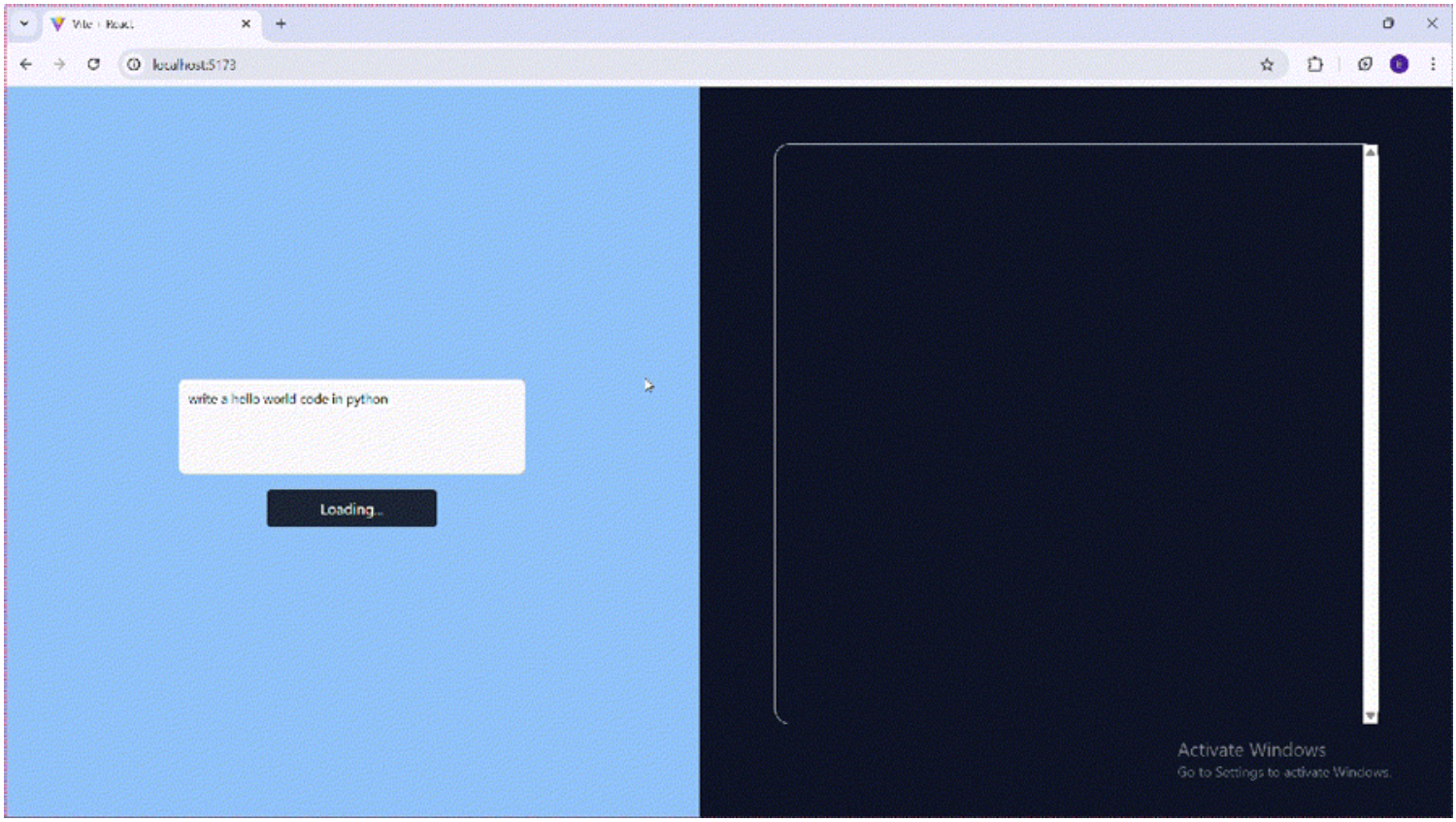
You'll learn how to:

- Set up `llm-ui` in your project
- Customize the LLM stream with blocks and components
- Implement a fallback block for unmatched content
- Compare `llm-ui` with similar tools

To bring it all together, we'll build a Code Viewer App that streams code output from Google's Gemini API, detects syntax, and renders it using syntax-highlighted code blocks.

By the end, you'll be able to create your own interactive LLM-powered interface, fully customized, cleanly styled, and ready for production.

Here's what it will look like:



Before you get started, make sure you have the following:

Would you be interested in joining LogRocket's developer community?

If you're evaluating LLM options for production, this guide comparing OpenAI and open-source models offers a helpful breakdown of trade-offs.

🚀 Sign up for The Replay newsletter

The Replay is a weekly newsletter for dev and engineering leaders.

Delivered once a week, it's your curated guide to the most important conversations around frontend dev, emerging AI tools, and the state of modern software.

Email

Submit

Why use llm-ui

- **Built-in support for common formats** – `llm-ui` natively handles Markdown, JSON, and CSV out of the box
- **Extensible with custom components** – You can define your own blocks to render virtually any kind of structured output
- **Works with any LLM API** – Since `llm-ui` operates on streamed output, it's compatible with any LLM provider, including OpenAI and Gemini

Project setup

We'll initialize our React app using Vite, a fast build tool optimized for modern frontend development. For styling, we'll use Tailwind CSS.

To get started, open your terminal, navigate to your project directory, and run:

Would you be interested in joining LogRocket's developer community?

N

```
npm install tailwindcss @tailwindcss/vite
```

Next, change the content of the vite.config.js file to this:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

Finally, delete the content of the index.css file and paste the following:

```
@import "tailwindcss";
```

Install llm-ui

To install `llm-ui`, run the following:

```
npm install @llm-ui/react @llm-ui/markdown react-markdown remark-gfm @llm-ui
```

NB: When running the above command, if you get an error like below, just downgrade your React version to version 18.0.0:

```
C:\Users\user\Desktop\coder>npm install @llm-ui/react @llm-ui/markdown react-markdown remark-gfm @llm-ui/code shiki html-react
-parser
npm error code ERESOLVE
npm error ERESOLVE unable to resolve dependency tree
npm error
npm error While resolving: llm-ui@0.0.0
npm error Found: react@19.1.0
npm error node_modules/react
npm error   react@"^19.1.0" from the root project
npm error
npm error Could not resolve dependency:
npm error peer react@"^18.0.0" from @llm-ui/react@0.13.3
```

Would you be interested in joining LogRocket's developer community?

Core concepts

To understand how llm-ui works, it's important to first grasp a few key concepts:

- `Block` – Defines how `llm-ui` should detect and render a specific type of LLM output, such as code, markdown, or JSON
- `LLMOutputComponent` – A React component responsible for rendering the output of a matched block. It receives structured data (`blockMatch`) from the `useLLMOutput` Hook. Common examples include `CodeBlock` and `Markdown`
- `useLLMOutput` **hook** – Listens to the LLM's streaming output, matches patterns based on defined blocks, and returns structured data for rendering
- `blockMatch` – An object containing the parsed output from the LLM stream. This is passed to the output component to determine how the content should be displayed
- `fallbackBlock` – A fallback strategy used when the stream doesn't match any defined block. Typically renders the unmatched content as Markdown using a `lookBack` function.
- `lookBack` **function** – Processes previously streamed content to determine what should be displayed and how much of it should be visible during the stream
- `throttle` – Controls the pacing of the stream animation, simulating a more natural, character-by-character output experience

Building our code viewer app

To keep our project well-organized, we'll separate concerns into three main folders:

- **blocks** – Contains configuration for detecting and rendering different types of LLM output (e.g., code blocks, Markdown)
- **ui** – Contains reusable React components for the user interface
- **utils** – Houses utility functions, API logic, and configuration files (e.g., for syntax

Would you be interested in joining LogRocket's developer community?

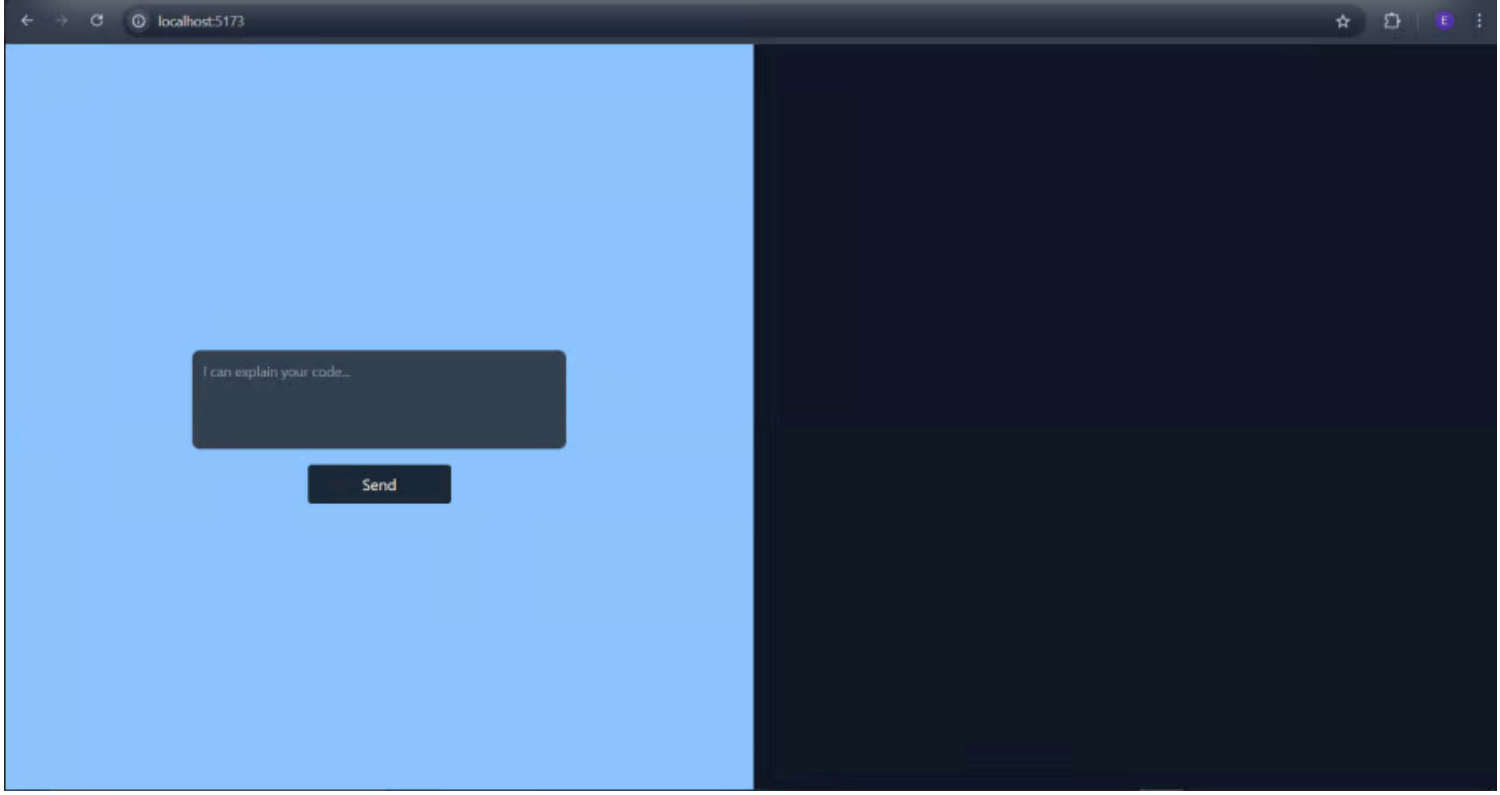
S

We have a simple user interface that we will use in this tutorial:

```
function App() {
  return (
    <div className="flex h-screen">
      <div className="flex-1/2 bg-blue-300 flex justify-center items-center"
        <form className="flex flex-col justify-between items-center w-full">
          <textarea
            rows={4}
            id="prompt"
            placeholder="I can explain your code..."
            className="block p-2.5 w-1/2 text-sm text-gray-900 bg-gray-50 rc
          />
          <button
            type="submit"
            className="bg-gray-800 text-amber-50 px-14 py-2 rounded-[4px] mt
          >Send</button>
        </form>
      </div>
      <div className="flex-1/2 flex justify-center items-center text-gray-1(
    </div>
  );
}
export default App;
```

Below is the output:

Would you be interested in joining LogRocket's developer community?



We will later move it to another file and integrate it with our app.

Setting up blocks

Now that the project is scaffolded, let's define the blocks that will handle different types of LLM output.

For this tutorial, we'll create two blocks:

- **Code block** – To detect and render syntax-highlighted code snippets
- **Markdown block** – A fallback block for rendering all other content as styled Markdown

The Markdown block will serve as the default display format when no specific match is found.

Inside the blocks folder, create a new file named `codeBlockBlock.jsx`, and start by importing the required utilities:

```
import {
  codeBlockLookBack,
```

Would you be interested in joining LogRocket's developer community?

In the above, we imported the LookBack function and the matchers that we will use in the `codeBlockBlock` configuration object.

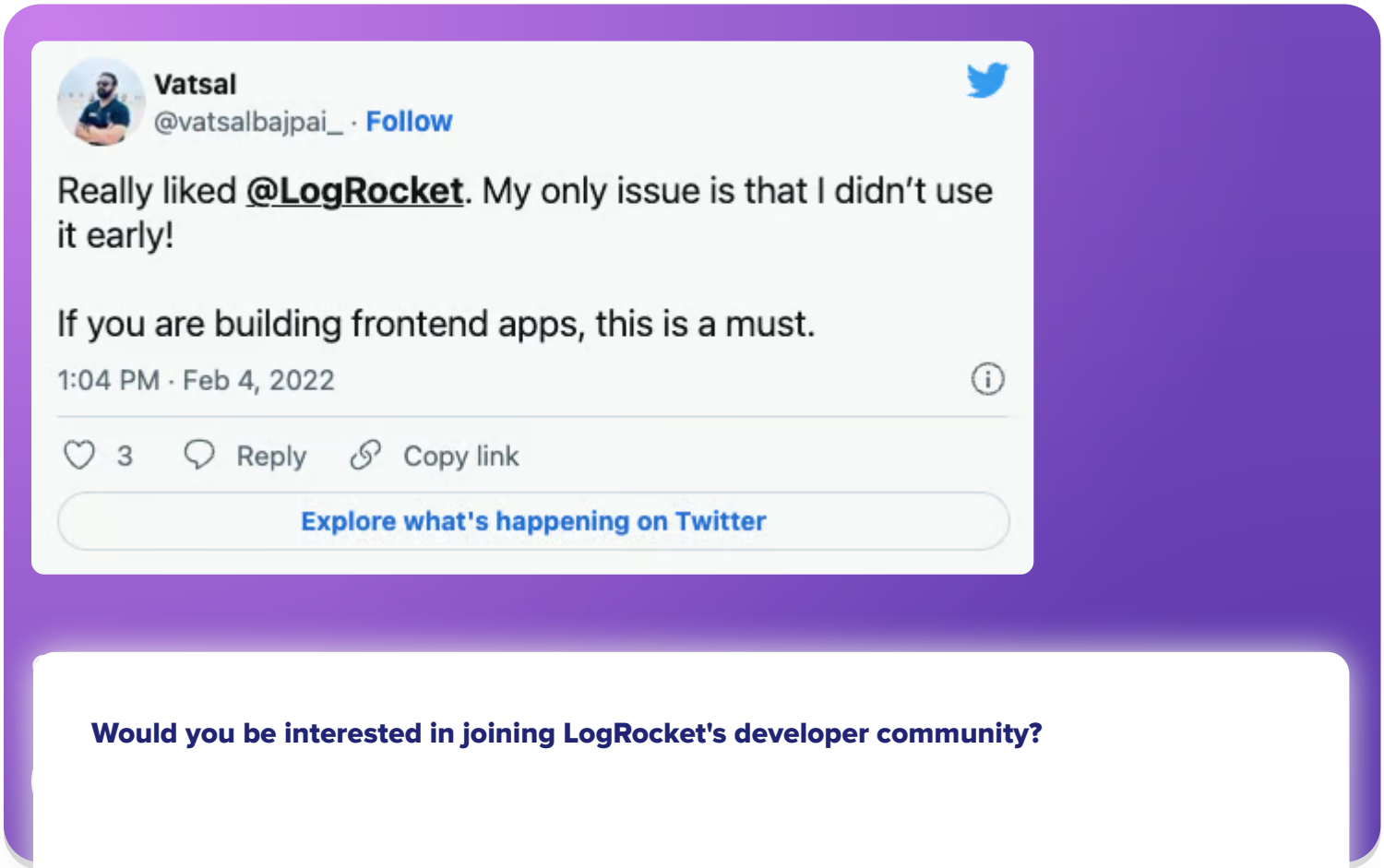
Next, we create the codeBlockBlock configuration object:

```
export const codeBlockBlock = {
  findCompleteMatch: findCompleteCodeBlock(),
  findPartialMatch: findPartialCodeBlock(),
  lookBack: codeBlockLookBack(),
};
```

The codeBlockBlock configuration object above contains the matcher functions and the lookback function. We then parse the configurations to these functions, respectively.

We'll use Shiki, a powerful syntax highlighter that supports over 100 languages and themes, including GitHub Dark and Light. It helps render beautifully styled code blocks with minimal configuration.

We need to create `shikiBlockComponent` for Shiki syntax highlighting in our code block.



Get into the utils folder and create a `shikiBlockComponent.jsx` file and paste the below code:

```
import { parseCompleteMarkdownCodeBlock } from "@llm-ui/code";
export const ShikiBlockComponent = ({ blockMatch }) => {
  const { code, language } = parseCompleteMarkdownCodeBlock(blockMatch.outpu
  if (!code) {
    return undefined;
  }
  return (
    <CodeBlock
      className="my-4"
      code={code}
      codeToHtmlOptions={{ lang: language }}
    />
  );
};
```

The `ShikiBlockComponent` is the presentation layer for the `codeBlockBlock` definition, taking the code content and rendering it using the Shiki syntax highlighter.

Let's get back to the codeBlockBlock.jsx file and import the `shikiBlockComponent.jsx` file.

We will also add `shikiBlockComponent` to the `codeBlockBlock` object.

`codeBlockBlock.jsx` will now look like this:

```
import {
  codeBlockLookBack,
  findCompleteCodeBlock,
  findPartialCodeBlock,
} from "@llm-ui/code";
import { CodeBlock } from "../ui/codeBlockUi";
```

Would you be interested in joining LogRocket's developer community?

```
  component: CodeBlock,
};
```

In summary, `codeBlockBlock` provides the `useLLMOutput` Hook with the rules it needs to detect, parse, and render code blocks from an ongoing text stream.

Setting up the fallback block

Next, we will set up the Markdown block, which serves as the fallback block

Create a `markdownBlock.jsx` file and paste the following code:

```
import ReactMarkdown from "react-markdown";
import remarkGfm from "remark-gfm";

const MarkdownComponent = ({ blockMatch, ...props }) => {
  const markdown = blockMatch.output;
  return (
    <ReactMarkdown
      {...props}
      remarkPlugins={[...(props.remarkPlugins ?? []), remarkGfm]}
    >
      {markdown}
    </ReactMarkdown>
  );
};
```

In the code above, we imported `ReactMarkdown`, a React library that converts Markdown into HTML, and `remark-gfm`, a plugin that enables GitHub-flavored Markdown support such as tables, strikethroughs, and task lists.

We then created a `MarkdownComponent`, which accepts a `blockMatch` prop (from the `useLLMOutput` hook) and renders the Markdown content using `ReactMarkdown`.

Would you be interested in joining LogRocket's developer community?

M

- Don't miss a moment with The Replay, a curated newsletter from LogRocket
- Learn how LogRocket's Galileo AI watches sessions for you and proactively surfaces the highest-impact things you should work on
- Use React's useEffect to optimize your application's performance
- Switch between multiple versions of Node
- Discover how to use the React children prop with TypeScript
- Explore creating a custom mouse cursor with CSS
- Advisory boards aren't just for executives. Join LogRocket's Content Advisory Board. You'll help inform the type of content we create and get access to exclusive meetups, social accreditation, and swag

---

Next, let's look at the second part of the code, a wrapper component that customizes how certain elements, like `<pre>` blocks, are rendered:

```
export const Markdown = (props) => {
  return (
    <MarkdownComponent
      {...props}
      components={{
        pre: ({ children }) => <pre>{children}</pre>,
      }}
    />
  );
};
```

The `Markdown` component acts as a wrapper around `MarkdownComponent`. It forwards all incoming `props` and allows you to customize how specific Markdown elements are rendered. In this case, it overrides the default rendering of `<pre>` tags(used for preformatted text like code blocks) to enable syntax highlighting and apply consistent styling.

Would you be interested in joining LogRocket's developer community?

S

In

```
import { allLangs, allLangsAlias, loadHighlighter } from "@llm-ui/code";
import { getHighlighterCore } from "shiki/core";
import { bundledLanguagesInfo } from "shiki/langs";
import githubDark from "shiki/themes/github-dark.mjs";
import githubLight from "shiki/themes/github-light.mjs";
import getWasm from "shiki/wasm";

export const shikiConfig = {
  highlighter: loadHighlighter(
    getHighlighterCore({
      langs: allLangs(bundledLanguagesInfo),
      langAlias: allLangsAlias(bundledLanguagesInfo),
      themes: [githubLight, githubDark],
      loadWasm: getWasm,
    }),
  ),
  codeToHtmlOptions: { themes: { light: "github-light", dark: "github-dark"
};
```

In the above, we imported all the necessary files for configuring Shiki.

We then use the `highlighter: loadHighlighter(...)` property to define and set the configuration for Shiki.

Rendering and managing the syntax highlighter

Navigate to the ui folder and create a `codeBlockUi.jsx` file. Then copy and paste the code below:

```
import { shikiConfig } from "../utils/shikiConfig";
import { useCodeBlockToHtml } from "@llm-ui/code";
import parseHtml from "html-react-parser";
```

Would you be interested in joining LogRocket's developer community?

```
    codeToHtmlOptions: {
      ...shikiConfig.codeToHtmlOptions,
    },
  });

  if (!html) {
    return (
      <pre className="shiki">
        <code>{code}</code>
      </pre>
    );
  }

  return <>{parseHtml(html)}</>;
};
```

This component is responsible for rendering and syntax highlighting of the code block.

It receives a `blockMatch`, then it uses the `useCodeBlockToHtml` hook and `shikiConfig` to generate syntax-highlighted HTML.

`Block` display component

Next, create a `blockToShow.jsx` file, and then get the imports like so:

```
import { codeBlockBlock } from "./blocks/codeBlock";
import { Markdown } from "./blocks/markdownBlock";
import { markdownLookBack } from "@llm-ui/markdown";
import { throttleBasic, useLLMOutput } from "@llm-ui/react
```

In the above imports, we made the necessary imports, notably the code block and Markdown block we created earlier.

N

Would you be interested in joining LogRocket's developer community?

```
    llmOutput: stream,
    fallbackBlock: {
      component: Markdown,
      lookBack: markdownLookBack(),
    },
    blocks: [codeBlockBlock],
    isStreamFinished,
    throttle: throttleBasic({ targetBufferChars: 60 }),
  });
```

In the code above, we created a functional component that receives two props: `stream` and `isStreamFinished`. We then destructure `blockMatches` from the `useLLMOutput` hook, passing the LLM stream to the `llmOutput` property.

The `fallbackBlock` defines how to render any unmatched text, in this case, using the `Markdown` component. The `blocks` array includes our custom `codeBlockBlock`, which handles code-specific output.

The `isStreamFinished` prop indicates whether the streaming is complete, while the `throttle` option controls the animation speed of the rendered text, simulating a more natural typing effect.

Here's the final part of the component, where we map over each `blockMatch` and render its corresponding component:

```
  return (
    <div className="flex flex-1 flex-col min-w-0 b border-gray-200  border-1
      {blockMatches.map((blockMatch, index) => {
        const Component = blockMatch.block.component;
        return <Component key={index} blockMatch={blockMatch} />;
      })}
    </div>
  );
```

Would you be interested in joining LogRocket's developer community?

G

Now let's set up the API call to Gemini to fetch streaming output from the LLM.

First, install the Gemini SDK by running:

```
npm i @google/generative-ai
```

Next, let's set up the environmental variable. Create a `.env` file in the root folder of this project and paste the following:

```
VITE_GOOGLE_API_KEY=My-Environmental-Variable
```

Now, let's set up the API request. In the "utils" folder, create a `geminiApi.jsx` component, then import the following:

```
import { useState } from "react";
import { GoogleGenerativeAI } from "@google/generative-ai";
```

Inside the component, create a GeminiApi function passing in `setResponseText` as props.

Thereafter, declare states like below:

```
const [prompt, setPrompt] = useState("");
const [isLoading, setIsLoading] = useState(false);
```

We will use the prompt state to hold our LLM input and `isLoading` for the button.

Next, we import the API key from the environmental variable we created earlier:

```
const GEMINI_API_KEY = import.meta.env.VITE_GOOGLE_API_KEY;
const genAI = new GoogleGenerativeAI(GEMINI_API_KEY)
```

We then create an instance of the `GoogleGenerativeAI` SDK, passing in the API key.

N

Would you be interested in joining LogRocket's developer community?

```
    setResponseText("");
    setIsLoading(true);

    console.log("Sending prompt:", prompt);

    try {
      const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });
      const result = await model.generateContentStream(prompt);

      let text = "";
      for await (const chunk of result.stream) {
        const chunkText = chunk.text();
        text += chunkText;
        setResponseText(text);
      }
    } catch (error) {
      console.error("Error generating content:", error);
      setResponseText(
        "Error: " + (error.message || "An unknown error occurred.")
      );
      if (error.response) {
        console.error("API Error Response:", error.response);
      }
    } finally {
      setIsLoading(false);
    }
  };

  }
```

The `handleSubmit` function sends the user's prompt to the Gemini 1.5 Flash model and streams the response back. As the output arrives, it updates the `setResponseText` state passed in via props. The function also handles errors gracefully and manages a loading state to provide feedback in the UI.

Would you be interested in joining LogRocket's developer community?

The Best of React on PodRocke

Preview Emily Kochanek

Save on Spotify

1	The React Core team talks React Forget and the future PodRocket	40:49
2	Why I still choose React with Chance Strickland PodRocket	36:03
3	The future of React with Theo Browne PodRocket	31:32

Next, we'll integrate the form-based user interface we created earlier into the main app component.

To learn more about Google's Gemini and its evolving capabilities, check out this deep dive on Gemini 2.5 and how it compares to other LLMs for frontend use cases.

Integrating the user interface

Now, let's integrate the UI we created earlier into the app.

Move the form element from the `App.js` component to be returned by the `GeminiApi` function, like so:

```
return (
  <form
    className="flex flex-col justify-between items-center w-full"
    onSubmit={handleSubmit}
  >
```

Would you be interested in joining LogRocket's developer community?

```
      placeholder="I can explain your code..."
      className="block p-2.5 w-1/2 text-sm text-gray-900 bg-gray-50 rounded-
    />
    <button
      type="submit"
      className="bg-gray-800 text-amber-50 px-14 py-2 rounded-[4px] mt-4 cur
      disabled={isLoading}
    >
      {isLoading ? "Loading..." : "Send"}
    </button>
  </form>
);
```

We also connected the form's `onSubmit` event to the `handleSubmit` function and used an `onChange` handler to keep the input field in sync with the `prompt` state.

Finally, update the content of `App.jsx` to the following:

```
import { useState } from "react";
import GeminiApi from "./utils/geminiApi";
import { BlockToShow } from "./blockToShow";

function App() {
  const [responseText, setResponseText] = useState("");

  return (
    <div className="flex h-screen">
      <div className="flex-1/2 bg-blue-300 flex justify-center items-center"
        <GeminiApi setResponseText={setResponseText} />
      </div>
      <div className="flex-1/2 flex justify-center items-center text-gray-10
        <BlockToShow stream={responseText} />
      </div>
    </div>
  );
```

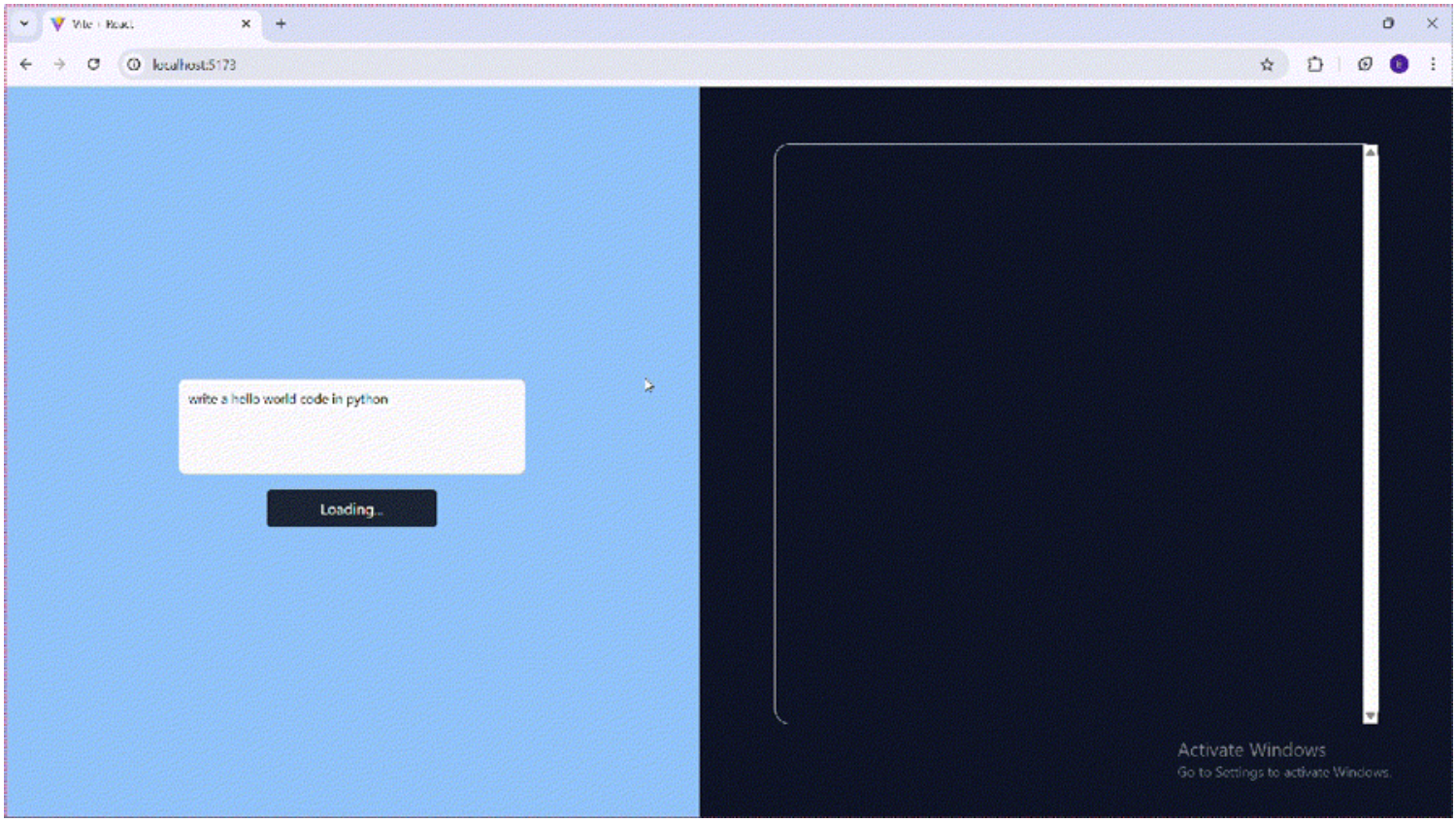
Would you be interested in joining LogRocket's developer community?

W

Next, the response is sent through `responseText` to the `BlockToShow` component.

We can run the code with `npm run dev`.

There you have it; our app recognizes code in an LLM stream and displays it using the code block. It then displays the rest of the stream using the markdown block:



Comparing llm-ui and its alternatives

Let's quickly take a look at a brief comparison of similar tools to `llm-ui`:

Tool	Description	Key Features	Ideal For
llm-ui	**A React-based UI framework for building custom LLM-powered generative interfaces**	◦ **Built-in chat components** ◦ **React-friendly hooks** ◦ **Themeable UI** ◦ **Easily swappable LLM backends**	◦ **Adding custom components into the llm output** ◦ **Smoothing out pauses in the LLM's response** ◦ **Building generative**

Would you be interested in joining LogRocket's developer community?

Tool	Description	Key Features	Ideal For
		◦ Syntax highlighter ◦ React Server Components support	◦ Building Context-Aware AI Assistants ◦ Building generative ui
Vercel's AI SDK UI	A set of framework-agnostic hooks for building chat and generative UIs	◦ Streaming responses ◦ React Server Components Support ◦ Edge-optimized UI ◦ Image generation	◦ Building AI agents ◦ Handling image and files generation
LangChain UI	A no-code, open source chat AI toolkit built on top of LangChain with Next.js, Chakra UI, Prisma and NextAuth	◦ Chatbot theming ◦ Prompt template ◦ Supports Auth provider of choice ◦ Supports all database	◦ Building complex, workflow-heavy AI agents. ◦ Building custom ChatGPTs like Chatbot

Conclusion

`llm-ui` makes it easy to transform raw LLM output into clear, structured, and user-friendly interfaces. In this tutorial, we walked through how to get started with `llm-ui`, explored its core concepts, and demonstrated how to customize the LLM stream, set up a fallback block, connect to a real LLM API like Gemini, and handle streaming responses in real time. We also compared `llm-ui` to other alternatives in the space.

If you're building an LLM-powered application, `llm-ui` is a powerful tool to help you add structure, flexibility, and polish to your AI interfaces.

Get set up with LogRocket's modern React error tracking in minutes:

Would you be interested in joining LogRocket's developer community?

2. Install LogRocket via npm or script tag. `LogRocket.init()` must be called client-side, not server-side

npm | Script tag

```
$ npm i --save logrocket

// Code:

import LogRocket from 'logrocket';
LogRocket.init('app/id');
```

3. (Optional) Install plugins for deeper integrations with your stack:
    - Redux middleware
    - NgRx middleware
    - Vuex plugin

Get started now

#react

LogRocket
Frontend Analytics

Stop guessing about your digital experience with LogRocket

Get started free

Would you be interested in joining LogRocket's developer community?

Recent posts:

Node.js project architecture best practices

Understand best practices for structuring Node.js projects, such as separating roles using folder structures and practicing modular code.

Piero Borrelli
Mar 20, 2026 · 16 min read

TypeScript at scale in 2026: What senior engineers should know

Would you be interested in joining LogRocket's developer community?

The Replay (3/18/26): Hiring in the AI era, coding isn't dead, and more

Discover what's new in The Replay, LogRocket's newsletter for dev and engineering leaders, in the March 18th issue.

Matt MacCormack
Mar 18, 2026 · 29 sec read

Thinking beats coding: How to hire the right engineers in the AI era

Would you be interested in joining LogRocket's developer community?

Mar 18, 2026 · 4 min read

View all posts

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

☐ **Save my name, email, and website in this browser for the next time I comment.**

Post Comment

Would you be interested in joining LogRocket's developer community?